

Mysterious Parody Bits

Lab 1 - COMP211 - Fall 2026

“In solving a problem of this sort, the grand thing is to be able to reason backwards. That is a very useful accomplishment, and a very easy one, but people do not practise it much. In the every-day affairs of life it is more useful to reason forwards, and so the other comes to be neglected. There are fifty who can reason synthetically for one who can reason analytically. . . Let me see if I can make it clearer. Most people, if you describe a train of events to them, will tell you what the result would be. They can put those events together in their minds, and argue from them that something will come to pass. There are few people, however, who, if you told them a result, would be able to evolve from their own inner consciousness what the steps were which led up to that result. This power is what I mean when I talk of reasoning backwards, or analytically.” ~ Sherlock Holmes, *A Study in Quiet*

Part 3 of 3. What goes parity must come ytirap

To resolve the final clue of this lab, you will write a program to decode the encoding of `parity.c`. Your source code program should be named `ytirap.c`, of course.

Not only should this program decode the output of `parity.c`, but it should also use the parity bit to detect whether any input bytes are corrupt or improperly encoded.

If a corrupted byte is found:

- Immediately print `\nCorruption detected!\n` to `stderr`
- Stop processing further input
- Exit with `EXIT_FAILURE`

Any successfully decoded characters before the corruption are printed to `stdout`.

Example Usage:

```
$ echo "12345" | ./parity | ./ytirap
12345
$ echo "12345" | ./parity
cefi# Expected, properly encoded parity string
$ echo "ceaij" | ./ytirap # Notice an 'a' was snuck in at index 2 (0-based indexing)
```

Corruption Detected!

```
12$
$ echo "cefi#j" | ./ytirap # Correct parity encoding of 12345
12345
```

Create a new file named `ytirap.c` in the root directory of your lab. Then implement `ytirap` as specified above.

A note about newlines - If you read a newline from the input, ignore it. Do not decode it and do not check its parity. Simply continue to the next input character. - Do not add a newline to the output. Only print characters that were successfully decoded from the input.

Modifying our Makefile

Modify your Makefile to add a `ytirap` target that builds the `ytirap` executable from `ytirap.c`. Update the `all` and `clean` targets accordingly. Then, answer the corresponding questions on Gradescope.

Testing

In Part 3, testing is a little different than before. Your program now has two kinds of outcomes:

1. Good input
 - Prints the decoded message to `stdout`
 - Prints nothing to `stderr`
 - Exits with `EXIT_SUCCESS` (0)
2. Corrupted input
 - Prints decoded characters up to the point of corruption to `stdout`
 - Prints `Corruption detected!` to `stderr`
 - Exits with `EXIT_FAILURE` (1)

To test this correctly, we need to check three things for each test case:

- what was printed to `stdout`
- what was printed to `stderr` (usually nothing)
- the program's exit code

This is a real-world concept: programs signal success or failure using exit codes.

File naming convention for tests

For each test case named `case`, create these files:

- `tests/ytirap/input/case.in` — input bytes for `ytirap`
- `tests/ytirap/expected/case.out` — expected `stdout`
- `tests/ytirap/expected/case.err` — expected `stderr` (often empty)
- `tests/ytirap/expected/case.code` — expected exit code (0 or 1)

Example Tests

Test 1: Good input (should succeed)

This uses your pipeline to generate a correct parity-encoded string:

```
printf "12345" | ./parity > tests/ytirap/input/good.in
```

Expected `stdout` is the decoded message:

```
printf "12345" > tests/ytirap/expected/good.out
```

Expected `stderr` is empty:

```
: > tests/ytirap/expected/good.err
```

Expected exit code is 0:

```
printf "0" > tests/ytirap/expected/good.code
```

Testing the good input

Run `ytirap`, redirecting `stdout` and `stderr` into separate files:

```
./ytirap < tests/ytirap/input/good.in \  
  > tests/ytirap/actual/good.out \  
  2> tests/ytirap/actual/good.err
```

Immediately after the program finishes, check its exit code:

```
echo $?
```

For this test, you should see:

```
0
```

Now compare the actual output files with the expected output files:

```
learncli$ diff tests/ytirap/actual/good.out tests/ytirap/expected/good.out
learncli$ diff tests/ytirap/actual/good.err tests/ytirap/expected/good.err
```

No output from either diff means those files match.

Finally, compare the exit code with the expected exit code. Since `echo $?` only works immediately after the command you want to inspect, it is useful to save it in a variable:

```
learncli$ ./ytirap < tests/ytirap/input/good.in > tests/ytirap/actual/good.out 2> tests/ytirap/actual/good.err
learncli$ code=$?
```

Now you can compare that value with the expected code:

```
learncli$ echo "$code"
learncli$ cat tests/ytirap/expected/good.code
```

Both should print:

```
0
```

Test 2: Corrupted input (should fail)

The correct encoding of 12345 is `cefi j`. If we “sneak in” an `a` at index 2 (0-based), corruption should be detected.

Create the corrupted input file:

```
echo -n "ceaij" > tests/ytirap/input/corrupt.in
```

Expected `stdout` (decoded characters before corruption, plus newline):

```
printf "12\n" > tests/ytirap/expected/corrupt.out
```

Expected `stderr` (error message):

```
printf "\nCorruption detected!\n" > tests/ytirap/expected/corrupt.err
```

Expected exit code: 1

```
printf "1" > tests/ytirap/expected/corrupt.code
```

Repeat the same process with the corrupted test:

```
learncli$ ./ytirap < tests/ytirap/input/corrupt.in > tests/ytirap/actual/corrupt.out 2> tests/ytirap/actual/corrupt.err
learncli$ code=$?
```

Check the exit code:

```
learncli$ echo "$code"
```

For corrupted input, you should see:

```
1
```

Then compare `stdout`:

```
learncli$ diff tests/ytirap/actual/corrupt.out tests/ytirap/expected/corrupt.out
```

Compare `stderr`:

```
learncli$ diff tests/ytirap/actual/corrupt.err tests/ytirap/expected/corrupt.err
```

And compare the exit code with the expected value:

```
learncli$ echo "$code"
```

```
learncli$ cat tests/ytirap/expected/corrupt.code
```

Both should print:

```
1
```

The Mystery: Solved

Once your implementation of `ytirap` is working, you can decode the final clue in `clues/03-doughnuts.hex`. Remember, you can output the contents of this file with `cat` before piping it into your decoder(s). It was first encoded with `parity`, and then encoded with `hex`. Once you successfully decode it, answers lie within.

Grading

You must include the course header in all `.c` files. Style is manually graded by course staff, after the late deadline, on the following criteria:

- All magic numbers are defined as constants
- No global variables
- For all parts, only bitwise operations were used in the conversions